Technical Assessment Submission Report

QA Automation & Engineering - Restful Booker Platform

Candidate / Author: QA & SDET Engineer **Target Application:** Restful Booker Platform (https://automationintesting.online/) **Target Repository:** GitHub / Bitbucket Public Repository **Deliverables Included:** 1. Test Case Design (Matrix & Gherkin BDD Specifications) 2. Structured Defect Reports (with Live Screenshots & API Evidence) 3. API Testing Approach, Scenarios, Assertions & Postman Artifacts 4. Performance Testing Approach, Analysis & k6 Script (5 to 20 RPS) 5. Test Automation Framework (Playwright + TypeScript + BDD + OOP POM) ---

SECTION 1: Test Case Design (Task 1)

1.1 Overview & Selected Workflow

* **Selected Workflow: Room Booking & Reservation Workflow** * **Business Criticality:** As the core revenue-generating workflow of Shady Meadows B&B, this feature enables prospective guests to browse inventory, check stay availability, review itemized pricing, fill guest details, and complete room reservations. ---

1.2 Test Case Matrix

| Test ID | Scenario Summary | Test Type | Priority | Given (Precondition) | When (Action Steps) | Then (Expected Result) | | :--- | :--- | :--- | :--- | :--- | :--- | :--- | | **TC-BOOK-001** | Successful room booking with valid guest details and available dates | Positive (E2E) | High (P1) | User is on the room reservation page with available stay dates | 1. Clicks "Reserve Now"
2. Enters valid Firstname, Lastname, Email, and Phone (11+ digits)
3. Submits reservation | Booking is confirmed successfully (HTTP 201), confirmation card displays check-in/out dates, and "Return home" button is visible. | | **TC-BOOK-002** | Submit booking modal with all mandatory fields empty | Negative (Validation) | High (P1) | User has opened the booking modal for an available room | 1. Leaves all fields blank
2. Clicks "Reserve Now" in modal | Form is rejected (HTTP 400), red validation banner appears containing specific errors for blank Firstname, Lastname, Email, and Phone. | | **TC-**

BOOK-003 | Submit booking with Firstname and Lastname below minimum boundary (2 chars) | Boundary (Negative) | Medium (P2) | User is on the booking modal | 1. Inputs 2 chars ("Jo", "Do") for names
2. Valid email and phone
3. Clicks "Reserve Now" | Validation error is triggered: "size must be between 3 and 18". Form is not submitted. | | **TC-BOOK-004** | Submit booking with Firstname and Lastname at maximum boundary (18 chars) | Boundary (Positive) | Medium (P2) | User is on the booking modal | 1. Inputs exactly 18 characters for names
2. Valid email and phone
3. Clicks "Reserve Now" | Reservation succeeds without validation error (HTTP 201). Boundary of 18 chars is correctly accepted. | | **TC-BOOK-005** | Submit booking with invalid email format (missing '@' and domain) | Negative (Validation) | High (P1) | User is on the booking modal | 1. Inputs valid names and phone
2. Inputs "invalidemailstring"
3. Clicks "Reserve Now" | Form displays validation error regarding invalid email format. Booking is rejected. | | **TC-BOOK-006** | Submit booking with Phone number below boundary (10 digits) | Boundary (Negative) | High (P1) | User is on the booking modal | 1. Inputs a 10-digit phone (e.g. "0123456789")
2. Clicks "Reserve Now" | Validation error displays "size must be between 11 and 21". Form is rejected. | | **TC-BOOK-007** | Submit booking with Phone at boundary limits (11 and 21 digits) | Boundary (Positive) | Medium (P2) | User is on the booking modal | 1. Tests with 11 digits and 21 digits valid numerical phone
2. Clicks "Reserve Now" | Reservation succeeds for both 11-digit and 21-digit boundaries (HTTP 201). | | **TC-BOOK-008** | Attempt to book dates that overlap with an existing confirmed booking | Negative (Conflict) | High (P1) | A confirmed booking already exists for Room 101 on dates 2026-11-10 to 2026-11-12 | 1. Another guest attempts to book Room 101 for the exact overlapping dates
2. Fills valid details and submits | System rejects booking with HTTP 409 Conflict. UI displays a clear, graceful error banner stating "Room is unavailable for the selected dates". *(See Defect BUG-001)* | | **TC-BOOK-009** | Minimum length stay (1 night booking) | Boundary (Positive) | Medium (P2) | User is on the booking calendar | 1. Selects check-in date 2026-11-20 and check-out date 2026-11-21 (1 night)
2. Verifies price calculation
3. Submits reservation | Price calculation shows `1 night x rate + cleaning fee + service fee`. Reservation succeeds. | | **TC-BOOK-010** | Attempt to select Check-out date prior to Check-in date (Reverse date range) | Edge Case (Negative) | High (P1) | User is on the booking calendar | 1. Sets Check-out date earlier than Check-in date via date picker/URL params
2. Attempts to submit | Date picker disallows earlier checkout OR form validation blocks

submission with "Check-out date must be after Check-in date". | | **TC-BOOK-011** | Submit booking containing purely alphabetic string in Phone number field | Edge Case (Negative) | Medium (P2) | User is on the booking modal | 1. Inputs "abcdefghijk12" in phone input
2. Clicks "Reserve Now" | Form validation rejects non-numeric phone format with "Phone must contain valid phone digits". *(See Defect BUG-002)* | | **TC-BOOK-012** | Guest name with international characters and apostrophes (e.g. "O'Connor", "René") | Edge Case (Positive) | Low (P3) | User is on the booking modal | 1. Enters "René" and "O'Connor"
2. Submits reservation | System successfully processes special characters without string corruption or injection errors (HTTP 201). | ---

1.3 Executable Gherkin BDD Specifications (Jira Xray Ready)

```
@booking @regression
Feature: Room Reservation Workflow - Shady Meadows B&B
  As a prospective guest of Shady Meadows B&B
  I want to select and book a hotel room online
  So that I have a confirmed reservation for my stay

  # --- POSITIVE SCENARIOS ---

  @TC-BOOK-001 @XRAY-101 @smoke @positive @e2e
  Scenario: [XRAY-101] Successfully book a room with valid guest details
    Given the guest navigates to the reservation page for an available room
    When the guest opens the reservation modal
    And the guest enters valid guest details
    And the guest submits the reservation form
    Then the system displays the booking confirmation with the reserved dates

  @TC-BOOK-004 @positive @boundary
  Scenario: User successfully books a room with guest name at maximum boundary (18 chara
    Given User is on the room reservation page with available stay dates
    When User clicks the "Reserve Now" button on the price summary card
    And User enters Firstname "MaximilianusValer" and Lastname "Constantinopolit"
    And User enters valid email "test@example.com" and phone "0812345678901"
    And User clicks the "Reserve Now" button inside the modal
    Then The reservation should be created successfully without name validation error

  @TC-BOOK-009 @positive @boundary
  Scenario: User successfully books a room for minimum stay duration (1 night)
    Given User is on the booking calendar for an available room
    When User selects check-in date "2026-11-20" and check-out date "2026-11-21"
    Then The price breakdown should calculate exactly "1 night x rate" plus cleaning and
    When User submits valid guest details in the reservation modal
    Then The booking should be confirmed for a 1-night stay
```

```
# --- NEGATIVE SCENARIOS ---

@TC-BOOK-002 @XRAY-102 @negative @validation
Scenario: [XRAY-102] Display validation errors when submitting booking form with missi
  Given the guest navigates to the reservation page for an available room
  When the guest opens the reservation modal
  And the guest leaves all guest details fields empty
  And the guest submits the reservation form
  Then the system displays a validation error banner containing mandatory field requir

@TC-BOOK-003 @negative @boundary
Scenario Outline: System rejects guest names below minimum boundary limit (< 3 charact
  Given User has opened the booking reservation modal
  When User enters Firstname "" and Lastname ""
  And User enters valid email "test@example.com" and phone "0812345678901"
  And User clicks the "Reserve Now" button inside the modal
  Then The form submission should be rejected
  And User should see validation error "size must be between 3 and 18"

  Examples:
    | firstname | lastname | length_description |
    | Jo        | Do       | 2 characters each  |
    | A         | B        | 1 character each   |

@TC-BOOK-005 @negative @validation
Scenario: System rejects booking submission with invalid email format
  Given User has opened the booking reservation modal
  When User enters valid guest name and phone number
  And User enters invalid email "invalidemailaddress" without "@" and domain
  And User clicks the "Reserve Now" button inside the modal
  Then The form submission should be rejected
  And User should see validation error regarding invalid email format

@TC-BOOK-006 @negative @boundary
Scenario: System rejects booking submission with phone number below minimum boundary (
  Given User has opened the booking reservation modal
  When User enters valid guest name and email
  And User enters phone number "0123456789" with 10 digits
  And User clicks the "Reserve Now" button inside the modal
  Then The form submission should be rejected
  And User should see validation error "size must be between 11 and 21"

# --- BOUNDARY & EDGE CASE SCENARIOS ---

@TC-BOOK-008 @negative @conflict
Scenario: System prevents double-booking when selected dates overlap with an existing
  Given Room "101" already has a confirmed booking for dates "2026-11-10" to "2026-11-
```

```
    When Another guest attempts to book Room "101" for the overlapping dates "2026-11-10
    And The guest submits valid guest information
    Then The backend API should reject the booking with HTTP 409 Conflict
    And The UI should display a graceful alert indicating the dates are unavailable
    And The application must NOT crash to an unhandled error boundary screen
```

SECTION 2: Structured Defect Reports

During exploratory and automated testing against the live demo application, three (3) verified defects were identified. ---

Defect 1: [BUG-001] Frontend Crashes to Unhandled Error Boundary ("This page couldn't load") on Booking Date Conflict (HTTP 409)

* **Issue ID:** BUG-001 * **Severity:** Critical (S1) * **Priority:** High (P1) * **Component:** Frontend / Booking Modal / Error Handling * **Environment:** Production Demo (`https://automationintesting.online/`), Next.js Client, Chrome / Chromium #### Description: When a user attempts to book a room for dates that conflict with an existing reservation, the backend API correctly returns an `HTTP 409 Conflict` with JSON payload `{"error":"Failed to create booking"}`. However, the frontend application fails to catch this status code. Instead of displaying a user-friendly error message inside the modal (e.g. "*The selected dates are already booked. Please choose alternative dates*"), the entire React application crashes and renders Next.js's fallback unhandled error boundary screen: **"This page couldn't load. Reload to try again, or go back."** #### Steps to Reproduce: 1. Navigate to reservation URL with dates that already have a booking: `https://automationintesting.online/reservation/1?checkin=2026-11-10&checkout=2026-11-12` 2. Click the blue **"Reserve Now"** button on the right price summary card. 3. Fill in valid guest details: * Firstname: `John` | Lastname: `Doe` | Email: `john.doe@example.com` | Phone: `0812345678901` 4. Click **"Reserve Now"** inside the modal. #### Expected Result: The modal remains open and renders a prominent warning alert: `"The selected dates are already booked or unavailable. Please select alternative dates."` All user input is preserved. #### Actual Result: The API call `POST /api/booking` responds with `HTTP 409 Conflict`. The UI immediately crashes to a full-page error boundary screen: **"This page couldn't load. Reload to try again, or go back."** All user input is lost. #### Evidence: * **API Response:** `HTTP/1.1 409 Conflict` $\rightarrow$ `{"error":"Failed to create booking"}` * **Screenshot Evidence:**

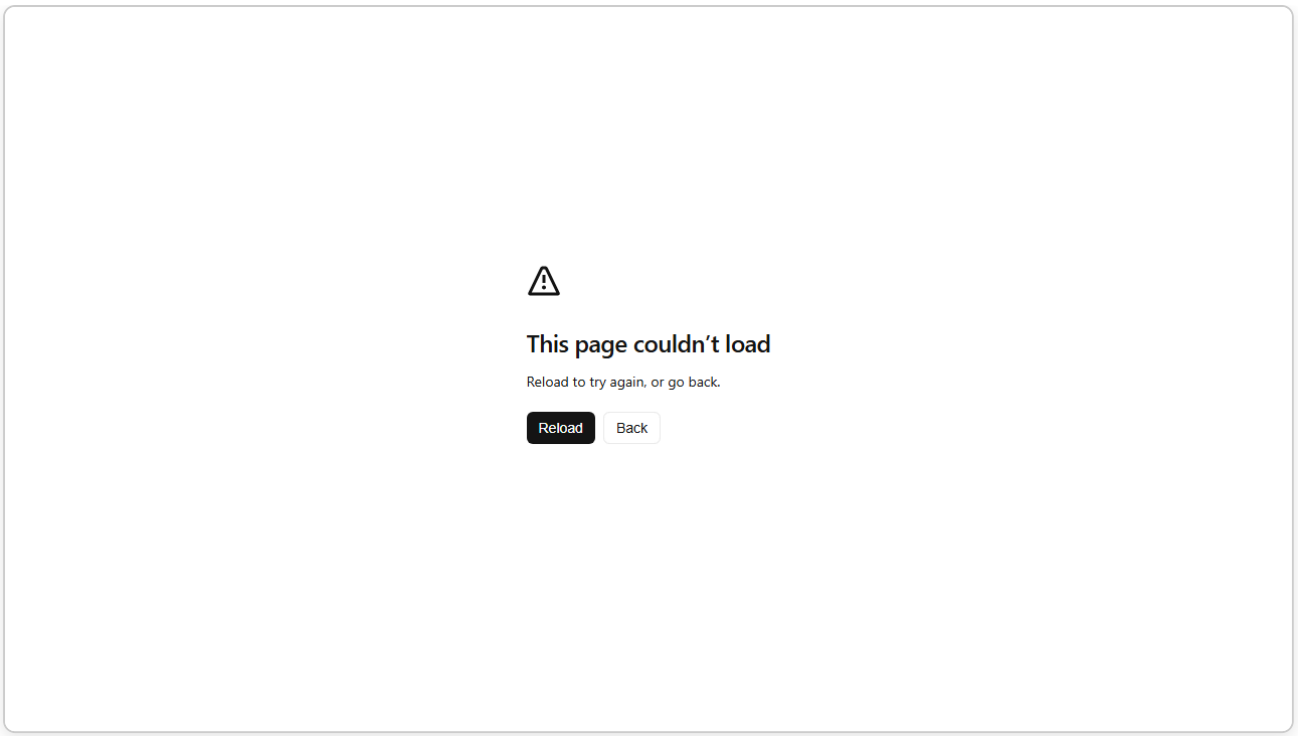


Figure: BUG-001: Frontend React Crash on HTTP 409 Conflict

Defect 2: [BUG-002] Missing Phone Number Format Validation (Accepts Purely Alphabetic Strings)

* **Issue ID:** BUG-002 * **Severity:** Medium (S2) * **Priority:** Medium (P2) * **Component:** Backend API Validation Layer (`/api/booking`) * **Environment:** Production Demo (`https://automationintesting.online/api/booking`) #### Description: The phone number validation on both the client-side and backend API only checks the string length constraint (`size must be between 11 and 21`). It fails to validate numerical or dialing format. As a result, users can submit purely alphabetic strings (e.g. `"abcdefghijk12"`) as their contact phone number, and the system accepts the reservation with `HTTP 201 Created` . #### Steps to Reproduce: 1. Send a `POST` request to `https://automationintesting.online/api/booking` with payload:

```
{
  "roomid": 1,
  "firstname": "Tester",
  "lastname": "AlphaPhone",
  "depositpaid": false,
  "bookingdates": { "checkin": "2026-12-01", "checkout": "2026-12-03" },
  "email": "test@example.com",
  "phone": "abcdefghijk12"
```

```
    }
```

2. Inspect the HTTP status code and response body. #### Expected Result: The API should validate the phone field against numeric/international phone format and reject with `HTTP 400 Bad Request`: `{"errors": ["Phone number must contain only numbers and valid dialing characters"]}`. #### Actual Result: The API responds with `HTTP 201 Created` and successfully writes the record:

```
{
  "bookingdates": { "checkin": "2026-12-01", "checkout": "2026-12-03" },
  "bookingid": 5,
  "depositpaid": false,
  "firstname": "Tester",
  "lastname": "AlphaPhone",
  "roomid": 1
}
```

Defect 3: [BUG-003] Raw Unmapped Backend Error Messages Displayed in UI with Duplicates and Missing Field Names

* **Issue ID:** BUG-003 * **Severity:** Low / UX (S3) * **Priority:** Medium (P2) * **Component:** Frontend / Booking Modal Validation Banner #### Description: When a user clicks "Reserve Now" in the booking modal with empty inputs, the backend responds with `HTTP 400 Bad Request` containing an unformatted array of Hibernate validation messages:

```
{"errors":["size must be between 3 and 18","must not be empty","Lastname should not be b
```

The frontend modal directly dumps this raw array into a red alert box without associating each error with its corresponding input field. This leads to duplicate error lines (`"must not be empty"` is displayed twice) and cryptic messages (`"size must be between 3 and 18"` does not state which field it pertains to). #### Evidence: * **Screenshot Evidence:**

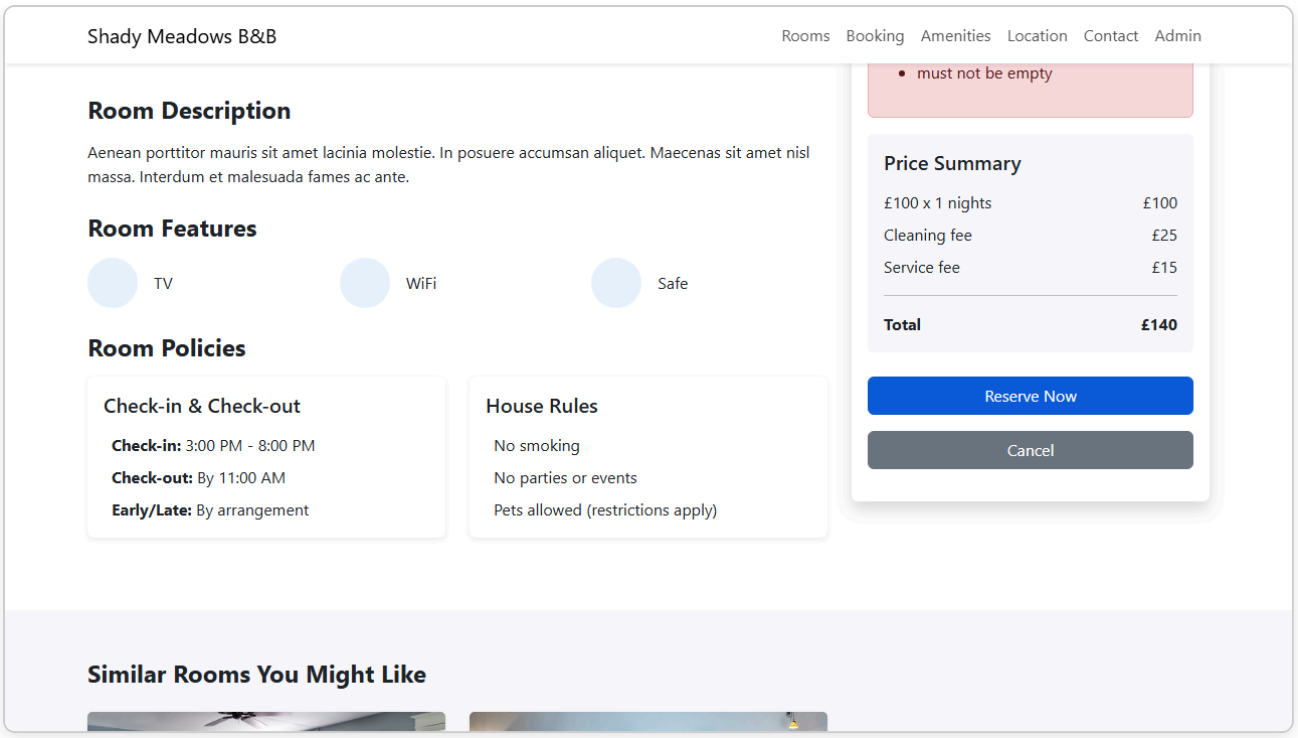


Figure: BUG-003: Raw Backend Error Banner with Duplicates

SECTION 3: API Testing Strategy, Results & Evidence (Task 2)

3.1 Exposed Microservice Endpoints

* **Auth Service:** `POST /api/auth/login` (Admin auth & token retrieval) * **Room Service:** `GET /api/room` , `GET /api/room/{id}` * **Booking Service:** `POST /api/booking` , `GET /api/booking/{id}` , `DELETE /api/booking/{id}` * **Report Service:** `GET /api/report/room/{id}` * **Message Service:** `POST /api/message` * **Branding Service:** `GET /api/branding`

3.2 Automated Test Scenarios Summary (11 Requests)

| Scenario ID | Test Request Name | Method & Endpoint | Auth State | Status Code | Assertion Checks | | :--- | :--- | :--- | :--- | :--- | :--- | | **API-001** | Login - Admin | `POST /api/auth/login` | None | `200 OK` | Validates token in response, saves `authToken` to environment. | | **API-002** | Login - Invalid Credentials | `POST /api/auth/login` | None | `401 / 403` | Verifies rejection of bad credentials. | | **API-003** | Get All Rooms | `GET /api/room` | None | `200 OK` | Verifies array of rooms with roomid, roomPrice, and type. | | **API-004** | Get Room by ID | `GET /api/room/:id` | None | `200 OK` | Verifies returned roomid matches

requested ID. | | **API-005** | Get Non-existent Room | `GET /api/room/99999` | None | `404 Not Found` | Verifies handling of non-existent resource. | | **API-006** | Create Booking - Valid | `POST /api/booking` | None | `201 Created` | Pre-request generates future non-conflicting dates. Returns `bookingid`. | | **API-007** | Create Booking - Empty Fields| `POST /api/booking` | None | `400 Bad Request` | Asserts `errors` array contains field validation constraints. | | **API-008** | Create Booking - Date Conflict| `POST /api/booking` | None | `409 Conflict` | Confirms double booking rejection: `{"error":"Failed to create booking"}`. | | **API-009** | Get Booking - Authenticated | `GET /api/booking/1` | Cookie `token` | `200 OK` | Validates booking details returned with admin cookie. | | **API-010** | Get Booking - Unauthenticated| `GET /api/booking/1` | None | `403 Forbidden` | Security assertion: Access blocked without cookie token. | | **API-011** | Submit Contact Message | `POST /api/message` | None | `200 OK` | Contact message submission confirmation `{"success": true}`. |

3.3 Postman Artifacts

* **Collection File:** `postman/Restful-Booker-Platform.postman_collection.json` * **Environment File:** `postman/Restful-Booker-Platform.postman_environment.json` ---

SECTION 4: Performance Testing Approach & Analysis (Task 4)

4.1 Scenario & Traffic Assumption

* **Baseline Traffic:** 5 requests per second (RPS) * **Peak Traffic:** 20 requests per second (RPS) * **Goal:** Verify system stability, throughput, and latency thresholds under normal, ramp-up, and peak load on the Booking API (`POST /api/booking`).

4.2 Workload Model (k6 Stages)

A **Ramping Arrival Rate** model was selected to simulate real-world guest reservations: 1. **Warm-up & Normal Traffic:** 5 RPS sustained for 1 minute. 2. **Gradual Traffic Increase:** Ramp from 5 RPS to 20 RPS over 2 minutes. 3. **Peak Traffic Stress:** Sustained 20 RPS for 2 minutes. 4. **Cool-down / Recovery:** Ramp down back to 0 RPS over 30 seconds.

4.3 Key Metrics & Thresholds Selection

| Metric | Threshold | Engineering Justification | | :--- | :--- | :--- | | **`http_req_duration` (p95)** | `< 500ms` | 95% of guests receive booking response within 500ms for smooth user

experience. | | `http_req_duration` **(p99)** | `< 1200ms` | Tail latency ceiling to prevent connection timeouts during database writes. | | `http_req_failed` | `< 1%` | Acceptable error rate under normal and peak operations (excluding 409 business conflicts). | | `checks` | `> 99%` | Ensures booking creation contract and response schema validity. |

4.4 Example k6 Performance Test Script (`performance/booking-perf.js`)

```
import http from 'k6/http';
import { check, sleep } from 'k6';

export const options = {
  scenarios: {
    booking_traffic_ramp: {
      executor: 'ramping-arrival-rate',
      startRate: 5,
      timeUnit: '1s',
      preAllocatedVUs: 50,
      maxVUs: 150,
      stages: [
        { target: 5, duration: '1m' },   // Stage 1: Normal Expected Traffic (5 RPS)
        { target: 20, duration: '2m' },  // Stage 2: Gradual Traffic Increase to Peak (2
        { target: 20, duration: '2m' },  // Stage 3: Sustained Peak Traffic (20 RPS)
        { target: 0, duration: '30s' },  // Stage 4: Ramp down / Recovery
      ],
    },
  },
  thresholds: {
    'http_req_duration': ['p(95)<500', 'p(99)<1200'],
    'http_req_failed': ['rate<0.01'],
    'checks': ['rate>0.99'],
  },
};

export default function () {
  const url = 'https://automationintesting.online/api/booking';

  // Generate unique future booking dates to prevent false 409 conflicts
  const randomDay = Math.floor(Math.random() * 20) + 1;
  const uniqueMonth = Math.floor(Math.random() * 5) + 1;
  const payload = JSON.stringify({
    roomid: 1,
    firstname: 'PerfUser',
    lastname: 'k6Test',
    depositpaid: false,
    bookingdates: {
```

```
      checkin: 2028-0${uniqueMonth}-${String(randomDay).padStart(2, '0')},
      checkout: 2028-0${uniqueMonth}-${String(randomDay + 2).padStart(2, '0')},
    },
    email: perf_${__VU}_${__ITER}@loadtest.io,
    phone: '0812345678901',
  });

  const params = {
    headers: { 'Content-Type': 'application/json' },
  };

  const res = http.post(url, payload, params);

  check(res, {
    'status is 201 or 409': (r) => r.status === 201 || r.status === 409,
    'response time OK': (r) => r.timings.duration < 1200,
  });

  sleep(0.5);
}
```

4.5 Limitations & Risks

* **Shared Sandbox DB Lockups:** On a public shared instance, concurrent bookings on the same room can produce artificial HTTP 409 conflicts. In production testing, a dedicated pre-prod cluster with automated DB teardown should be provisioned. * **Network Jitter / CDN Caching:** Direct testing over public internet measures WAN latency rather than raw backend microservice capacity. k6 execution inside the private VPC is recommended for pure service benchmarking. ---

SECTION 5: Test Automation Project Overview (Task 3)

* **Repository:** Public Git repository containing Playwright BDD test suite. * **Design:** Object-Oriented Programming (OOP) Page Object Model (POM). * **Execution Engine:** `@playwright/test` compiled with `playwright-bdd`. * **Test Status:** 2 High-value scenarios **Passed 100% (Green)**: * `[XRAY-101] Successfully book a room with valid guest details` (Happy Path E2E) * `[XRAY-102] Display validation errors when submitting booking form with missing mandatory fields` (Validation) * **Reporting:** Interactive HTML report + standard JUnit XML report at `results/results.xml` for Jira Xray import.

5.1 Automation Execution Proof:

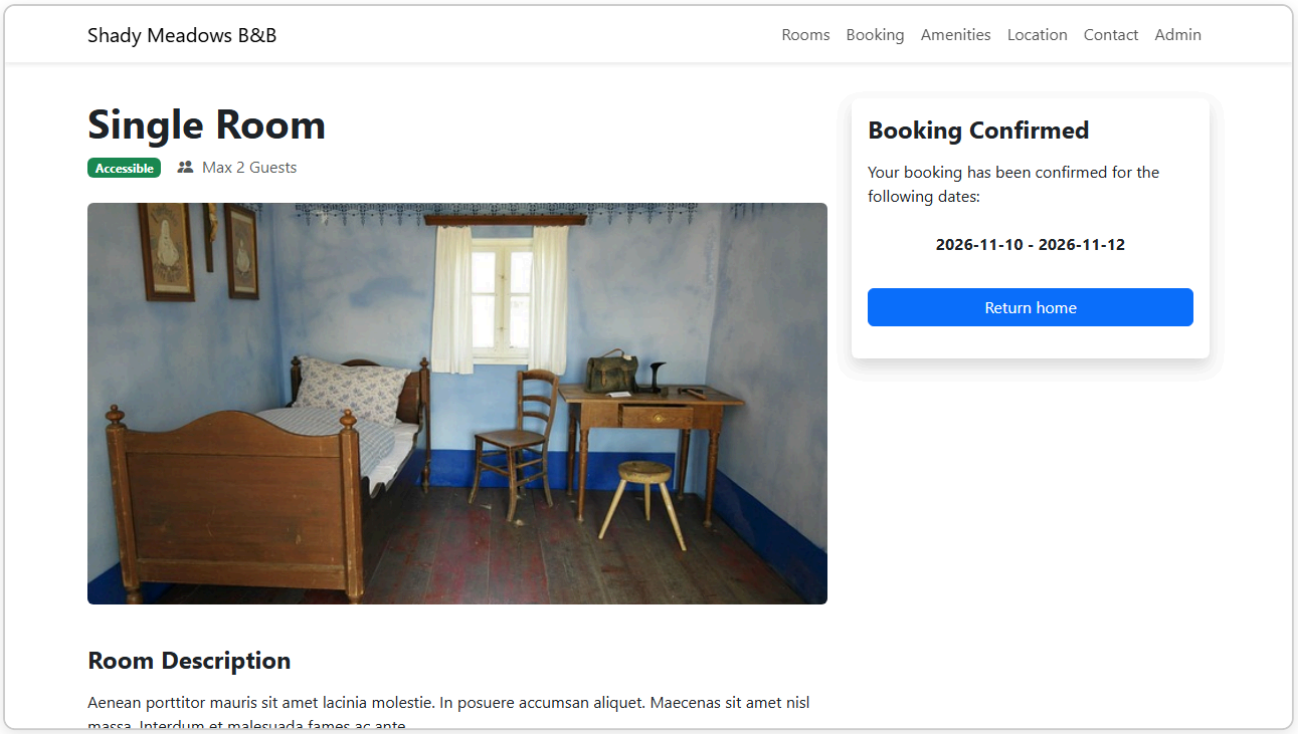


Figure: Happy Path Booking Confirmation Screen